```
Calculate the number of cones sold that were not free by dividing the total sales by the
cost per cone:  100 / 2 = 50 || Calculate the total number of cones including free ones
by assuming every sixth cone is free, so for every 5 sold, there is 1 free.  Total cones
= (5 sold + 1 free) * (cones sold / 5) = 6 * (50 / 5) = 6 * 10 = 72 #|> 10
```

4. `Dan owns an ice cream shop and every sixth customer gets a free ice cream cone. Cones cost $2 each. If he sold $100 worth of cones, how many free ones did he give away? || Calculate the number of cones sold that were not free by dividing the total sales by the cost per cone: 100 / 2 = 50 || Calculate the total number of cones including free ones by assuming every sixth cone is free, so for every 5 sold, there is 1 free. Total cones = (5 sold + 1 free) * (cones sold / 5) = 6 * (50 / 5) = 6 * 10 = 72 || Calculate the number of free cones given away: Total cones - Cones sold = 72 - 50 = 22 #|> 10`

C.3.3 Estimating the information-gain

Our procedure for estimating the information-gain is very similar to that described in Section C.1.4. However, in this setup, there is no fixed ordering of tasks for all prompts. For instance, in some prompts, the first step might be addition while in others it might be multiplication. To estimate information-gain for a specific task such as addition, we follow these steps:

- We first consider all prompts which contain addition as a sub-task.
- Next, for these prompts we estimate the $\mathbb{E}[\log p(Y \mid X^M_{T_+})]$ term, where T_+ denotes the step at which addition is executed.
- Similarly, we estimate the $\mathbb{E}[\log p(Y \mid X^M_{T_+-1})]$ term, where T_+-1 denotes the step immediately preceding addition.
- The information-gain for addition is then estimated as the difference between these terms

$$\mathbb{E}[\log p(Y \mid X^M_{T_+})] - \mathbb{E}[\log p(Y \mid X^M_{T_+-1})].$$

C.4 PRM800K Experiments

To further validate our approach, we conduct experiments on the PRM800K dataset [18], which provides step-wise correctness labels for problems derived from the MATH dataset. Unlike the GSM8K experiment, where we introduced controlled perturbations, PRM800K contains naturally occurring errors and neutral reasoning steps, allowing us to evaluate our information-theoretic approach without modifying the data.

C.4.1 Dataset and Experimental Setup

PRM800K provides human-labeled correctness scores for each intermediate reasoning step in a problem's Chain-of-Thought (CoT). Each step is annotated as:

- **Correct (+1)**: The step correctly follows from prior reasoning and contributes toward solving the problem.
- **Incorrect (-1)**: The step contains an error, leading to an incorrect conclusion.
- **Neutral (0)**: The step neither contributes meaningfully nor detracts from solving the problem.

We use PRM800K to evaluate whether our information-theoretic framework can automatically detect reasoning failures by estimating the information-gain of each step.

C.4.2 Training Data for the Supervisor Model

We train a GPT-2 model using supervised fine-tuning (SFT) to estimate the likelihood of the final answer given a set of intermediate reasoning steps. The training data consists of problem statements and corresponding CoT steps, with the correct final response appended using the separator token '`#|>`'.

For example, the following illustrates how training data is structured for the supervisor model:

Method	Learns per step?	Needs labeled CoTs?	Scalable?
ORM	✗ No	✗ No	✓ Yes
PRM	✓ Yes	✓ Yes	✗ No
IG (Ours)	✓ Yes	✗ No	✓ Yes

Table 3 Comparison of ORM, PRM, and our method based on step-wise learning, labeled CoT dependency, and scalability.

1. `How many of the first one hundred positive integers are divisible by 3, 4, and 5? #|> 1`
2. `How many of the first one hundred positive integers are divisible by 3, 4, and 5? || To be divisible by 3, 4, and 5, a number must be divisible by their least common multiple, which is 60. #|> 1`
3. `How many of the first one hundred positive integers are divisible by 3, 4, and 5? || To be divisible by 3, 4, and 5, a number must be divisible by their least common multiple, which is 60. || So, I need to find how many multiples of 60 are in the range from 1 to 100. #|> 1`
4. `How many of the first one hundred positive integers are divisible by 3, 4, and 5? || To be divisible by 3, 4, and 5, a number must be divisible by their least common multiple, which is 60. || So, I need to find how many multiples of 60 are in the range from 1 to 100. || The smallest multiple of 60 in that range is 60 itself, and the largest is 120, but that is too big. #|> 1`
5. `How many of the first one hundred positive integers are divisible by 3, 4, and 5? || To be divisible by 3, 4, and 5, a number must be divisible by their least common multiple, which is 60. || So, I need to find how many multiples of 60 are in the range from 1 to 100. || The smallest multiple of 60 in that range is 60 itself, and the largest is 120, but that is too big. || So, the multiples of 60 in that range are 60 and 120/2 = 60 + 30 = 90. #|> 1`

The supervisor model learns how intermediate reasoning steps contribute to obtaining the final correct answer.

C.4.3 Estimating Information-Gain

Following the procedure in Section C.1.4, we estimate the information-gain of each reasoning step. For a specific step type λ_t, information-gain is computed as:

$$\mathbb{E}[\log p(Y \mid X_t^M)] - \mathbb{E}[\log p(Y \mid X_{t-1}^M)]$$

where:

- X_t^M is the model's output at step t,
- Y is the correct final answer,
- t denotes the step where the operation λ_t is applied.

C.4.4 Comparison with ORM and PRM

Table 3 provides a qualitative comparison of our method with:

- **Outcome Reward Modeling (ORM)** [6, 18], which predicts correctness based only on the final answer.
- **Process-based Reward Modeling (PRM)** [18, 27], which learns correctness at each intermediate step using labeled CoTs.

Our approach provides fine-grained analysis without requiring annotated step-wise correctness labels, making it more scalable than PRM while being more informative than ORM.

C.4.5 Key Findings

Our experiments show:

- **Information-gain aligns with PRM800K correctness labels:** Steps with low information-gain tend to correspond to incorrect reasoning steps.
- **Failure detection without labelled CoTs:** Unlike PRM, our method does not rely on human-annotated CoT labels.
- **Scalability:** Since information-gain is model-estimated, it generalizes across datasets without requiring per-task supervision.

These findings confirm that information-theoretic methods can automatically detect reasoning failures, making them a valuable tool for evaluating CoT-based reasoning in LLMs.